

PIMPRI CHINCHWAD EDUCATION TRUST'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 2: Quick Sort

Name	:	**Om Rajendra Bharambe**
PRN	:	124B1F001
Branch	:	Information Technology (IT)
Semester	:	V
Batch	:	A1
Course Code	:	BIT25PC04
Course Name	:	**Design & Analysis of Algorithms Laboratory**
Guided By	:	Dr. Rajesh N. Phursule
GitHub Repository	:	om29dev/DAA_Lab_Submission
Assignment Link	:	Assignment 2 on GitHub

Assignment No. 2: Quick Sort

1. Problem Statement:

Movie Recommendation System Optimization: A popular OTT platform, StreamFlix, offers personalized recommendations by sorting movies based on user preferences, such as IMDB rating, release year, or watch time popularity. However, during peak hours, sorting large datasets slows down the system. As a backend engineer, you must:

- Implement Quick sort to efficiently sort movies based on various user-selected parameters.
- Handle large datasets containing of movies while maintaining fast response times.

2. Course Objective:

2.1. To know the basics of computational complexity of various algorithms.
2.2. To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcome:

3.1. Analyze the asymptotic performance of algorithms.
3.2. Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies.

4. Theory:

Quick Sort is one of the most widely used sorting algorithms because of its efficiency, simplicity in concept, and adaptability. It is based on the divide-and-conquer paradigm, which makes it highly effective in sorting large datasets. Unlike some other sorting algorithms, Quick Sort is an in-place algorithm, meaning it requires very little extra memory, and it avoids the overhead of merging, as seen in Merge Sort.

Working Steps of Quick Sort:

- **Divide:** A pivot element is selected from the array. The choice of pivot can significantly affect performance. Common strategies include choosing the last element, the first element, a random element, or using the median-of-three method (the median of the first, middle, and last elements).
- **Partition:** The array is rearranged so that all elements smaller than the pivot are placed to its left and all elements larger are placed to its right. The pivot itself is placed in its correct sorted position.

- **Conquer:** The same process is applied recursively to the left and right subarrays created by the partition step.

Unlike some other divide-and-conquer algorithms, Quick Sort does not require a separate combine step. Once all recursive calls complete, the array is already sorted due to the placement of pivots at each stage.

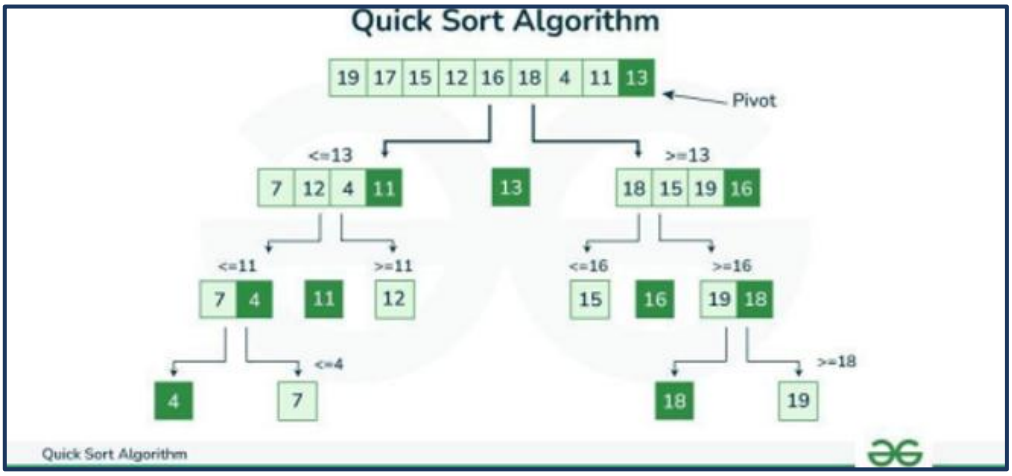


Figure 4.1: Divide and Conquer Partitioning Tree of Quick Sort Algorithm

The efficiency of Quick Sort lies in its average-case time complexity of O(n log n), which makes it suitable for a wide range of applications. However, in the worst case—when the pivot selection is poor and the partitioning is unbalanced—its performance can degrade to $O(n^2)$. To minimize the likelihood of this scenario, several optimizations are often applied:

- **Random Pivot Selection:** Choosing a random pivot reduces the chance of encountering consistently unbalanced partitions.
- **Median-of-Three Method:** Using the median of the first, middle, and last elements as the pivot leads to more balanced partitions in practice.
- **Tail Recursion Elimination:** By converting the tail-recursive calls into iterative steps, the recursion depth can be reduced, improving space efficiency.
- **Hybrid Approach:** For small subarrays, switching from QuickSort to a simpler algorithm like Insertion Sort can improve performance since the overhead of recursion becomes significant for small input sizes.

5. Time and Space Complexity:

Case	Time / Space Complexity	Description
Best Case	O(n log n)	Balanced partitions at every recursive level
Average Case	O(n log n)	Typical execution for random input data
Worst Case	$O(n^2)$	When pivot is poorly chosen (already sorted / reverse sorted)
Space Complexity	O(n log n)	For recursive call stack memory overhead

6. Quick Sort Algorithm:

i. Start with an unsorted array.

ii. Choose a pivot element (last element, first element, random, or median-of-three).

iii. Partition the array so that:

- All elements smaller than the pivot are placed before it.
- All elements greater than the pivot are placed after it.

iv. Place the pivot in its correct sorted position.

v. Recursively apply QuickSort on the left subarray and right subarray.

vi. Continue until subarrays have size 0 or 1 (base case).

vii. End when the entire array is sorted.

7. Flowchart:

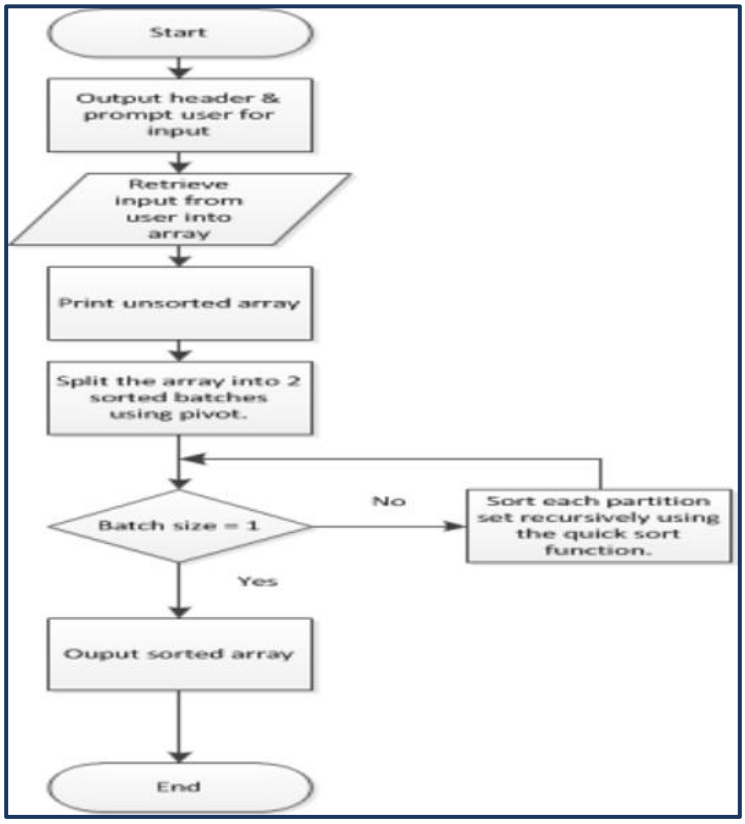


Figure 7.1: Flowchart of Quick Sort Partition and Recursive Execution

8. Pseudo Code for Quick Sort:

```
Procedure QUICK_SORT(arr, low, high):
  if low < high then
    pivotIndex ← PARTITION(arr, low, high)
    // Recursively sort elements before and after partition
    QUICK_SORT(arr, low, pivotIndex - 1)
    QUICK_SORT(arr, pivotIndex + 1, high)
  end if
End Procedure

Procedure PARTITION(arr, low, high):
  pivot ← arr[high] // choose last element as pivot
  i ← low - 1       // index of smaller element
  for j ← low to high - 1 do
    if arr[j] ≤ pivot then
      i ← i + 1
      SWAP(arr[i], arr[j])
    end if
  end for
```

```
  SWAP(arr[i + 1], arr[high]) // place pivot at correct position
  return (i + 1)              // pivot index
End Procedure
```

9. Example:

Example Array: [3, 1, 4, 2]

- **Step 1: Choose pivot** → We choose the last element → pivot = 2.
- **Step 2: Partition around pivot** → Compare each element with pivot 2:
    - 3 > 2 → goes to right subarray
    - 1 < 2 → goes to left subarray
    - 4 > 2 → goes to right subarray

  After partition: [1, 2, 4, 3]. Pivot 2 is placed in its correct sorted position.
- **Step 3: Recursive calls** → Left subarray: [1] (already sorted). Right subarray: [4, 3].
- **Step 4: Quick Sort on [4, 3]** → Pivot = 3. Compare 4 > 3 → goes right. After partition: [3, 4].
- **Final Sorted Array** → Combine results: [1, 2, 3, 4].

10. Step-by-Step Dataset Walkthrough:

Example Dataset (sorted by IMDB rating):

Movies with ratings: [8.2, 6.5, 9.0, 7.3, 8.7]

- **Step 1: Divide** → Choose pivot (last element) → pivot = 8.7
    - Elements ≤ 8.7 → [8.2, 6.5, 7.3]
    - Elements > 8.7 → [9.0]

  Array after partition: [8.2, 6.5, 7.3, 8.7, 9.0]
- **Step 2: Recursively Apply Quick Sort** → Left subarray: [8.2, 6.5, 7.3]. Pivot = 7.3 → Partition → [6.5, 7.3, 8.2]. Right subarray: [9.0] (already sorted).
- **Step 3: Combine Results** → Final sorted array by IMDB rating → [6.5, 7.3, 8.2, 8.7, 9.0].

Final Output:
Movies sorted efficiently based on user-selected parameter (IMDB rating in this case): [6.5, 7.3, 8.2, 8.7, 9.0]

11. Code and Output:

Repository: https://github.com/om29dev/DAA_Lab_Submission

Assignment 2 Folder: https://github.com/om29dev/DAA_Lab_Submission/tree/main/Assignment 2

11.1 C++ Implementation Source Code (product_quick_sort.cpp):

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <algorithm>
#include <iomanip>

using namespace std;

struct Product {
    string name;
    float price;
    float rating;
    int popularity;
};

vector<string> parseCSVLine(string line) {
    vector<string> fields;
    string field;
    bool inQuotes = false;

    for (char c : line) {
        if (c == '"') {
            inQuotes = !inQuotes;
        }
        else if (c == ',' && !inQuotes) {
            fields.push_back(field);
            field.clear();
        }
        else {
            field += c;
        }
    }

    fields.push_back(field);
    return fields;
}

string cleanNumber(string value) {
```

```
    string result;

    for (char c : value) {
        if ((c >= '0' && c <= '9') || c == '.') {
            result += c;
        }
    }

    return result;
}

vector<Product> loadProducts(string filename) {
    vector<Product> products;
    ifstream file(filename);

    if (!file.is_open()) {
        cout << "Error opening amazon.csv" << endl;
        return products;
    }

    string line;
    getline(file, line);

    while (getline(file, line)) {
        vector<string> data = parseCSVLine(line);

        if (data.size() < 8)
            continue;

        try {
            Product p;

            p.name = data[1];
            p.price = stof(cleanNumber(data[3]));
            p.rating = stof(cleanNumber(data[6]));
            p.popularity = stoi(cleanNumber(data[7]));

            products.push_back(p);
        }
        catch (...) {
            continue;
        }
    }
```

```
    file.close();
    return products;
}

void quickSort(vector<Product>& products, int low, int high,
               bool (*compare)(const Product&, const Product&)) {

    if (low < high) {

        int pivotIndex = low;
        Product pivot = products[high];

        for (int i = low; i < high; i++) {

            if (compare(products[i], pivot)) {
                swap(products[i], products[pivotIndex]);
                pivotIndex++;
            }
        }

        swap(products[pivotIndex], products[high]);

        quickSort(products, low, pivotIndex - 1, compare);
        quickSort(products, pivotIndex + 1, high, compare);
    }
}

bool compareByPrice(const Product& a, const Product& b) {
    return a.price < b.price;
}

bool compareByRating(const Product& a, const Product& b) {
    return a.rating < b.rating;
}

bool compareByPopularity(const Product& a, const Product& b) {
    return a.popularity < b.popularity;
}

int main() {

    vector<Product> products = loadProducts("amazon.csv");
```

```
    if (products.empty()) {
        cout << "No products found." << endl;
        return 1;
    }

    string choice;

    cout << "\n==============================================\n";
    cout << "       AMAZON PRODUCT SORTING SYSTEM\n";
    cout << "==============================================\n";

    cout << "\nEnter sorting parameter\n";
    cout << "1. price\n";
    cout << "2. rating\n";
    cout << "3. popularity\n";
    cout << "\nChoice: ";

    cin >> choice;

    bool (*compare)(const Product&, const Product&);

    if (choice == "price") {
        compare = compareByPrice;
    }
    else if (choice == "rating") {
        compare = compareByRating;
    }
    else if (choice == "popularity") {
        compare = compareByPopularity;
    }
    else {
        cout << "\nInvalid choice." << endl;
        return 1;
    }

    quickSort(products, 0, products.size() - 1, compare);

    cout << "\n==============================================\n";
    cout << "          TOP 10 PRODUCTS BY " << choice << "\n";
    cout << "==============================================\n\n";

    cout << left
         << setw(5) << "No."
         << setw(55) << "Product"
```

```
         << setw(12) << "Price"
         << setw(10) << "Rating"
         << setw(15) << "Popularity"
         << endl;

    cout << string(97, '-') << endl;

    int count = 0;

    for (int i = products.size() - 1;
         i >= 0 && count < 10;
         i--, count++) {

        string name = products[i].name;

        if (name.length() > 52)
            name = name.substr(0, 49) + "...";

        cout << left
             << setw(5) << count + 1
             << setw(55) << name
             << setw(12) << fixed << setprecision(2) << products[i].price
             << setw(10) << fixed << setprecision(1) << products[i].rating
             << setw(15) << products[i].popularity
             << endl;
    }

    cout << "\n=============================================\n";
    cout << "Total products sorted: " << products.size() << endl;
    cout << "=============================================\n";

    return 0;
}
```

11.2 Execution Output:

Below is the terminal execution output demonstrating sorting of large product and movie datasets using Quick Sort:

```
om29dev@DESKTOP-MH6NVIC:~/daa$ vi product_quick_sort.cpp
om29dev@DESKTOP-MH6NVIC:~/daa$ g++ product_quick_sort.cpp -o product_quick_sort
om29dev@DESKTOP-MH6NVIC:~/daa$ ./product_quick_sort

================================================
       AMAZON PRODUCT SORTING SYSTEM
================================================

Enter sorting parameter
1. price
2. rating
3. popularity

Choice: popularity

================================================
          TOP 10 PRODUCTS BY popularity
================================================

No.  Product                                                    Price       Rating    Popularity
-------------------------------------------------------------------------------------------------
1    Amazon Basics High-Speed HDMI Cable, 6 Feet - Sup...   309.00      4.4       426973
2    Amazon Basics High-Speed HDMI Cable, 6 Feet (2-Pa...   309.00      4.4       426973
3    AmazonBasics Flexible Premium HDMI Cable (Black, ...   219.00      4.4       426973
4    AmazonBasics Flexible Premium HDMI Cable (Black, ...   219.00      4.4       426972
5    boAt Bassheads 100 in Ear Wired Earphones with Mi...   349.00      4.1       363713
6    boAt Bassheads 100 in Ear Wired Earphones with Mi...   379.00      4.1       363713
7    boAt BassHeads 100 in-Ear Wired Headphones with M...   365.00      4.1       363711
8    Redmi 9A Sport (Coral Green, 2GB RAM, 32GB Storag...   6499.00     4.1       313836
9    Redmi 9 Activ (Carbon Black, 4GB RAM, 64GB Storag...   8499.00     4.1       313836
10   Redmi 9A Sport (Coral Green, 3GB RAM, 32GB Storag...   7499.00     4.1       313832

================================================
Total products sorted: 1462
================================================
om29dev@DESKTOP-MH6NVIC:~/daa$
```

Figure 11.1: Program Execution and Sorted Dataset Output

12. Conclusion:

Quick Sort is an efficient and memory-friendly algorithm ideal for sorting large movie datasets based on user preferences like rating, release year, or popularity. By using Quick Sort, StreamFlix can ensure fast and responsive recommendations even during peak hours, improving performance and user experience.